1. Ay: C Temelleri ve İlk Haberleşme Projesi

Oct 7, 2026 · @Canalp

Bu plan, haftada 10-12 saatlik çalışmayla dört haftada C temellerinden kendi haberleşme protokolünü çözen bir packet parser'a ulaşmayı hedefler. Tüm çalışmalar bilgisayar üzerinde yapılır.

Haftalık plan

Her hafta konular çalışılır, alıştırmalar yapılır ve hafta bir ödevle kapanır. Kurallar her hafta için aynı:

- Tüm kod tek bir GitHub reposunda: `hafta1/`, `hafta2/`, `hafta3/`, `proje1/` klasörleri
- Her klasörde nasıl derlenip çalıştırılacağını anlatan kısa bir README
- Her program `-Wall -Wextra -Werror` ile uyarısız derlenmeli
- Hafta sonunda 15-20 dakikalık kod incelemesi: kodu anlatır, "bu satır neden var?" sorularına cevap verir
- Sadece "çalışıyor" yetmez; hatalı girdiyle de denenmiş olmalı

1. Hafta: C temelleri ve araçlar

Haftanın sonunda kullanıcıdan güvenli girdi alan, menülü, fonksiyonlara bölünmüş bir programı Makefile ile derleyip gdb ile adım adım izleyebiliyor olmalı.

Konular

- Çalışma ortamı: Linux'ta ya da Windows'ta WSL (Ubuntu) ile `gcc`, `make`, `gdb`, `git`, `valgrind` kurulumu
- Derleme aşamaları: preprocessor (`gcc -E`), assembly (`gcc -S`), nesne dosyası (`gcc -c`), linkleme; basit bir `Makefile`
- Değişkenler ve veri tipleri: `int`, `char`, `float`, `double`; sabit genişlikli tipler `uint8_t`, `uint16_t`, `uint32_t`, `int32_t`
- `sizeof`, tip sınırları (`limits.h`, `stdint.h`), taşma; signed/unsigned karşılaştırma tuzakları
- if / else, switch, for / while / do-while
- Fonksiyonlar, parametre ve dönüş değeri, hata kodu döndürme
- `printf` biçim belirteçleri ve sabit genişlikli tipler için `inttypes.h` (`PRIu32`)
- Temel hata kontrolü ve güvenli girdi: `scanf` yerine `fgets` + `strtol`
- gdb ile breakpoint, `next`, `step`, `print`

Alıştırmalar

- Her tipin `sizeof` değerini ve sınırlarını yazdıran `tipler.c`
- Taşma deneyi: `uint8_t x = 250; x += 10;` sonucu neden 4?
- Sürpriz deneyi: `if (-1 < 1u)` neden yanlış çıkıyor?
- 1'den N'e toplamı for, while ve do-while ile üç ayrı fonksiyonda yazmak
- Bir sayının asal olup olmadığını döndüren fonksiyon
- Bu programlardan birini gdb'de adım adım çalıştırıp değişkenleri izlemek

Güvenli girdi alma kalıbı

`scanf("%d")` harf girildiğinde sessizce bozulur ve girdi buffer'da takılı kalır. Mesleki kodda satır `fgets` ile okunur, `strtol` ile sayıya çevrilir ve üç şey kontrol edilir: hiç rakam okundu mu, satırın sonunda fazladan karakter var mı, sayı taşması oldu mu (`errno == ERANGE`).

```
#include <errno.h>
#include <stdlib.h>

/* 0: başarılı, -1: geçersiz giriş */
int read_long(long *out)
{
    char line[64];
    char *end;

    if (fgets(line, sizeof line, stdin) == NULL) {
        return -1;
    }
    errno = 0;
    long value = strtol(line, &end, 10);
    if (end == line || errno == ERANGE) {
        return -1;
    }
    while (*end == ' ' || *end == '\n') {
        end++;
    }
    if (*end != '\0') {
        return -1;
    }
    *out = value;
    return 0;
}
```

Bu fonksiyonu ezberlemesi değil, her satırının neden orada olduğunu açıklayabilmesi beklenir.

Haftalık ödev: Ölçüm Konsolu

`do-while` ile dönen, `switch` ile seçim yapan bir menü programı:

- **1)** İki `uint16_t` sayıyı topla; sonuç 65535'i aşarsa taşma uyarısı ver.
- **2)** Sıcaklık sınıflandır: -40 ile 125 dışındaysa "sensör hatası", 0'ın altı "donma", 0-40 "normal", 40 üstü "yüksek".
- **3)** 1'den N'e toplam (N en fazla 100000, sonuç `uint32_t`).
- **4)** Bir sayının tek/çift ve asal olup olmadığını söyle.
- **0)** Çıkış.

Gereksinimler:

- Bütün girdiler `read_long` benzeri bir fonksiyonla alınır; "abc", boş satır, "99999999999" ve aralık dışı değerler programı çökertmez, "Geçersiz giriş" mesajı verir.
- Her menü işlemi ayrı bir fonksiyondadır; `main` sadece menüyü yönetir.
- `make` derler, `make clean` temizler.
- Git'te en az 5 anlamlı commit vardır.
- README'de gdb ile toplam fonksiyonuna breakpoint koyup değişkenleri adım adım izlediğini anlatan 3-4 satır not vardır.

Haftanın kontrol listesi

- [ ] gcc, make, gdb ve git kurulu ve çalışıyor
- [ ] Derleme aşamalarını anlatabiliyor
- [ ] `uint8_t` taşmasını ve `-1 < 1u` sürprizini açıklayabiliyor
- [ ] Ölçüm Konsolu hatalı girdilerle denendi ve çökmüyor
- [ ] gdb ile bir fonksiyonu adım adım izledi

2. Hafta: Veri yapıları ve sayı sistemleri

Haftanın sonunda verileri struct ve dizilerle düzenleyebilmeli, bir sayıyı bit ve bayt seviyesinde parçalayıp birleştirebilmeli. Haberleşmedeki her paket bu iki becerinin üzerine kurulur.

Konular

- Array, `char` ve ASCII tablosu, string ve sonlandırıcı `'\0'`; `strncpy` ve `snprintf` ile güvenli kopyalama
- `struct`, `enum`, `typedef`, `const`; struct dizileri
- Binary, hexadecimal, decimal ve aralarındaki dönüşümler; bir hex hanenin tam olarak 4 bite (nibble) karşılık gelmesi

- Byte (8 bit), word (16 bit), dword (32 bit). "Word" mimariye göre değişebildiği için kodda hep `uint16_t` gibi açık tipler kullanılır.
- Bitwise operatörler: `&`, `|`, `^`, `~`, `<<`, `>>`; maske ile bit set/clear/toggle/test
- Pointer'a giriş: `&` (adres) ve `*` (adresteki değer)

Alıştırmalar

- `'A'` ile `65`'in aynı şey olduğunu gösteren küçük bir ASCII tablosu yazdırıcı
- Kendi `my_strlen` ve `my_strcpy` fonksiyonları; sonlandırıcı unutulunca ne olduğunu denemek
- Sensör kaydı tasarımı: `typedef struct { uint8_t id; sensor_type_t type; int16_t temp_x10; } sensor_t;` ve 5 elemanlı bir dizinin yazdırılması (sıcaklığı 10 ile çarpılmış tam sayı olarak tutmak, float yerine yaygın bir yöntemdir)
- Kağıt üzerinde 10 sayıyı decimal, binary ve hex arasında çevirmek
- `set_bit`, `clear_bit`, `toggle_bit`, `test_bit` fonksiyonları
- 16 bitlik bir sayıyı yüksek ve düşük bayta ayırmak (`(v >> 8) & 0xFF`), sonra tekrar birleştirmek
- `&` ve `*` ile bir değişkenin adresini ve değerini yazdırmak, Python Tutor'da görselleştirmek

Bitlerle durum bayrakları örneği

Gerçek cihazlarda birçok açık/kapalı bilgi tek bir baytın bitlerinde tutulur:

```
enum {
    FLAG_READY = 0,   /* bit 0 */
    FLAG_ERROR = 1,   /* bit 1 */
    FLAG_BUSY  = 2    /* bit 2 */
};

uint8_t status = 0;
status |=  (1u << FLAG_READY);          /* set    */
status &= ~(1u << FLAG_BUSY);           /* clear  */
status ^=  (1u << FLAG_ERROR);          /* toggle */
if (status & (1u << FLAG_READY)) { }    /* test   */
```

Haftalık ödev: Bit Aracı (`bitaraci`)

Komut satırından bir sayı alıp onu her açıdan gösteren program:

```
$ ./bitaraci 0x1234
Decimal    : 4660
Hex        : 0x1234
Binary     : 0001 0010 0011 0100
High byte  : 0x12
Low byte   : 0x34
1 olan bit : 5
Bayt ters  : 0x3412
```

Gereksinimler:

- Sayı `argv[1]`'den alınır; hem `4660` hem `0x1234` kabul edilir (`strtoul`, taban 0).
- Binary gösterim kütüphane fonksiyonu olmadan, kaydırma ve maskeyle yazılır; 4'erli gruplanır.
- 1 olan bitleri sayan ve baytları ters çeviren fonksiyonlar kendi yazılır.
- Geçersiz girdide ("0xZZ", boş, 32 biti aşan sayı, argüman yok) kullanım bilgisi basıp hata koduyla çıkar.
- Test değerleri: `0`, `1`, `255`, `256`, `0xFFFF`, `0xA5A5`. Çıktıları README'de.

"Bayt ters" satırı bilinçli olarak var: 4. haftadaki endianness konusunun ön hazırlığı.

Haftanın kontrol listesi

- [ ] 0x2F, 200 ve 0b10110011'i kağıt üzerinde diğer iki tabana çevirebiliyor
- [ ] Bir bitin set, clear, toggle ve test ifadelerini mantığıyla yazabiliyor
- [ ] String'in sonlandırıcısı unutulunca ne olacağını açıklayabiliyor
- [ ] `bitaraci` tüm test değerlerinde doğru, hatalı girdide çökmüyor

3. Hafta: Pointer ve bellek yönetimi

C'nin güçlü ve tehlikeli tarafı bu hafta. Hedef pointer'ları ezberden değil, belleğin bir resmini kafada çizerek kullanabilmek ve bellek hatalarını araçlarla yakalayabilmek.

Konular

- Pointer, pointer aritmetiği, pointer'ın pointer'ı (kısaca)
- Pointer ve array ilişkisi: `arr[i]` ile `*(arr + i)` aynı şey; diziyi fonksiyona verince pointer'a dönüşmesi
- Pointer ve struct: `->` operatörü, struct'ı fonksiyona pointer ile vermek
- `NULL` ve her pointer'ı kullanmadan önce kontrol etme alışkanlığı
- Stack, heap ve static bellek bölgeleri; değişkenlerin ömrü
- `malloc`, `calloc`, `realloc`, `free`
- `sizeof`, struct padding ve `offsetof`

- Header/source ayrımı, header guard, `static` ve `extern`
- Hata yakalama araçları: AddressSanitizer (`-fsanitize=address`) ve `valgrind`

Alıştırmalar

- `swap(int *a, int *b)` fonksiyonu
- Bir dizinin toplamını indeks kullanmadan, pointer aritmetiğiyle hesaplamak
- `sizeof(dizi)` değerini `main` içinde ve dizinin verildiği bir fonksiyon içinde yazdırıp farkı açıklamak
- 2. haftadaki `sensor_t`'yi pointer ile dolduran bir fonksiyon
- Yerel bir değişkenin adresini döndüren hatalı bir fonksiyon yazıp neden bozulduğunu gdb ile görmek
- Bilerek dört hata içeren bir program: bellek sızıntısı, serbest bırakılmış belleği kullanma, iki kez `free`, dizinin bir fazlasına yazma. Önce AddressSanitizer, sonra `valgrind` ile çalıştırıp her raporun satırını bulmak.

Padding deneyi

```
struct a { uint8_t x; uint32_t y; uint8_t z; };   /* sizeof: 12 */
struct b { uint32_t y; uint8_t x; uint8_t z; };   /* sizeof: 8  */
```

Aynı alanlar, farklı sıra, farklı boyut. Derleyici `uint32_t`'yi 4'ün katı bir adrese hizalamak için araya boş baytlar koyar. Bu yüzden haberleşmede struct'ı `memcpy` ile doğrudan buffer'a kopyalamak yasak: karşı taraf bu boş baytları veri sanır. `offsetof` ile her alanın konumunu yazdırarak boşlukları kendi gözüyle görsün.

Haftalık ödev: Dinamik bayt dizisi (`bytebuf`)

Gerektikçe büyüyen bir bayt dizisi modülü. Ay sonu ödevinde paket encoder'ı çıktısını buraya yazabilir.

```
typedef struct {
    uint8_t *data;
    size_t   len;   /* kullanılan bayt */
    size_t   cap;   /* ayrılmış kapasite */
} bytebuf_t;

int  bytebuf_init(bytebuf_t *b, size_t initial_cap);
int  bytebuf_append(bytebuf_t *b, uint8_t byte);
int  bytebuf_append_many(bytebuf_t *b, const uint8_t *src, size_t n);
int  bytebuf_get(const bytebuf_t *b, size_t index, uint8_t *out);
void bytebuf_clear(bytebuf_t *b);
void bytebuf_free(bytebuf_t *b);
void bytebuf_print_hex(const bytebuf_t *b);
```

Gereksinimler:

- Kapasite dolunca `realloc` ile iki katına çıkar. `realloc` başarısız olursa eski veri kaybolmamalı (sonucu önce geçici bir pointer'a al).
- `NULL` argüman ve dizi dışı indeks durumunda fonksiyonlar `-1` döndürür, asla çökmez.
- `bytebuf_free` sonrası pointer `NULL` yapılır; iki kez çağrılması zararsızdır.
- Dosyalar: `bytebuf.h`, `bytebuf.c`, `main.c`, `test_bytebuf.c`.
- `make` programı, `make test` testleri, `make asan` sanitizer'lı sürümü derler.
- Testler: 1000 bayt ekleme, dizi dışı okuma, toplu ekleme, temizleyip yeniden kullanma.
- `valgrind` çıktısında "All heap blocks were freed" görülür.

Haftanın kontrol listesi

- [ ] Bir pointer kodunu kağıda kutu-ok diyagramı olarak çizebiliyor
- [ ] Stack ve heap farkını ve hangi değişkenin ne zaman yok olduğunu açıklayabiliyor
- [ ] Dört hatanın ASan ve valgrind raporlarını okuyup satırını bulabildi
- [ ] `bytebuf` testleri geçiyor, valgrind temiz

4. Hafta: Haberleşme altyapısı

Bu hafta ay sonu ödevinde kullanılacak dört yapı taşı hazırlanır: ring buffer, endianness fonksiyonları, CRC ve durum makinesi. Haftanın kalan zamanının tamamı ay sonu ödevine ayrılır.

Konular

- Buffer kavramı: sabit boyutlu buffer, sınır kontrolü, taşma
- Ring buffer (dairesel FIFO): `head`, `tail`, başa sarma, dolu ve boş durumunu ayırt etme
- Endianness: big-endian ve little-endian, "ağ bayt sırası" (big-endian)
- Checksum ve CRC farkı; CRC-16/MODBUS algoritması
- Durum makinesi: `enum` ile durumlar, `switch` ile geçişler, her girdiyle bir adım ilerleme

Alıştırmalar

- 8 baytlık ring buffer: `rb_push`, `rb_pop`, `rb_count`, `rb_is_full`; doldurma, taşırma, boşaltma ve başa sarma testleri. Ay sonu ödevinde gönderen ile alıcı arasındaki "hat" bu ring buffer olacak.
- `write_u16_be`, `read_u16_be`, `write_u32_be`, `read_u32_be` ve little-endian karşılıkları
- Bir `uint32_t`'nin ilk baytına bakarak kendi bilgisayarının little-endian mı big-endian mı olduğunu bulan program; `bitaraci`'daki "bayt ters" satırıyla bağlantısını kurmak

- Basit toplam checksum yazıp zayıflığını görmek: `12 34` ile `34 12` aynı toplamı verir, yani yer değiştirmiş baytları yakalayamaz. Ardından CRC-16/MODBUS yazıp aynı iki dizi için farklı sonuç verdiğini göstermek.
- Klavyeden karakter karakter gelen "ON" ve "OFF" kelimelerini bir durum makinesiyle tanıyan küçük program

CRC-16/MODBUS algoritması

1. `crc = 0xFFFF` ile başla.
2. Her bayt için: `crc ^= bayt`, ardından 8 kez tekrarla: en düşük bit 1 ise `crc = (crc >> 1) ^ 0xA001`, değilse `crc >>= 1`.
3. Sonuç `crc`. Gönderirken önce düşük bayt (`crc & 0xFF`), sonra yüksek bayt (`crc >> 8`).

Kontrol değeri: `"123456789"` dizisi için sonuç `0x4B37` olmalı. Bu değer tutmadan ay sonu ödevine geçilmez.

Haftalık ödev: Bu haftanın ödevi aşağıdaki ay sonu ödevidir.

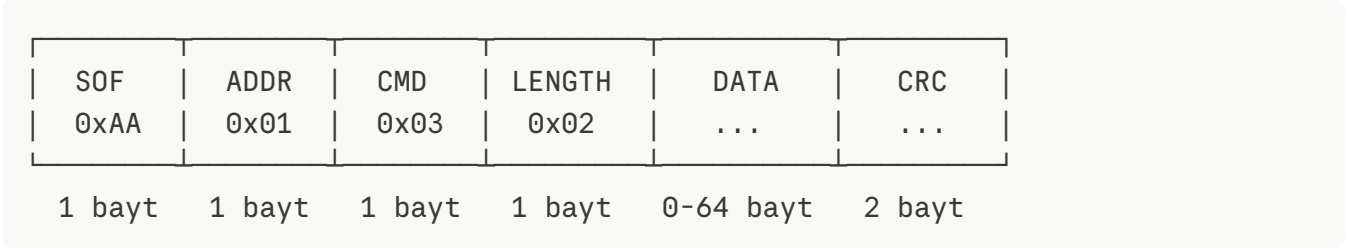
Haftanın kontrol listesi

- [ ] Ring buffer'ın dolu ve boş durumunu nasıl ayırt ettiğini açıklayabiliyor
- [ ] Kendi bilgisayarının bayt sırasını koduyla gösterdi
- [ ] Checksum'ın yakalayamadığı, CRC'nin yakaladığı bir örnek gösterebiliyor
- [ ] CRC fonksiyonu `0x4B37` kontrol değerini veriyor
- [ ] ON/OFF durum makinesi çalışıyor

Ay sonu ödevi föyü: Mini haberleşme protokolü ve packet parser

Tahmini süre 12-15 saat. Gerçek haberleşmede veri tek parça gelmez: bayt bayt gelir, bazen bozuk, bazen yarım, bazen başında çöp baytlarla. Bu projede kendi paket formatını tanımlayıp bu bayt akışından güvenilir şekilde paket çıkaran bir parser yazacak. 1-4. haftada yazdığı her şey (güvenli girdi, bit işlemleri, `bytebuf`, ring buffer, endianness, CRC, durum makinesi) burada birleşiyor.

Paket formatı



Alan	Boyut	Açıklama
SOF	1 bayt	Her zaman `0xAA`; paketin başladığını gösterir
ADDR	1 bayt	Hedef cihaz adresi, `0x01` – `0xFE`
CMD	1 bayt	Yapılacak işin kodu (komut tablosu)
LENGTH	1 bayt	DATA alanının bayt sayısı, en fazla 64
DATA	0-64 bayt	Komutun verisi; çok baytlı sayılar big-endian
CRC	2 bayt	CRC-16/MODBUS; ADDR'dan DATA sonuna kadar, SOF hariç; önce düşük bayt

En uzun paket 1 + 1 + 1 + 1 + 64 + 2 = 70 bayttır; alıcı buffer'ı buna göre boyutlanır.

Komut tablosu

CMD	Ad	DATA	Anlamı
`0x01`	PING	0 bayt	Cihaz cevap veriyor mu?
`0x02`	VERSION_READ	0 bayt	Yazılım sürümünü iste
`0x03`	VALUE_WRITE	2 bayt	16 bitlik bir değer yaz
`0x04`	VALUE_READ	0 bayt	Kayıtlı değeri iste

Tabloda olmayan bir CMD gelirse paket geçerli sayılır ama "bilinmeyen komut" hatası verilir.

Örnek paket

Gönderen program şu baytları üretir:

```
AA 01 03 02 12 34 B5 33
```

Alıcı program bunu şöyle çözümler:

```
SOF      = AA      → paket başlangıcı
ADDRESS  = 01      → 1 numaralı cihaz
COMMAND  = 03      → VALUE_WRITE
LENGTH   = 02      → 2 bayt veri var
DATA     = 12 34   → 0x1234 = 4660
CRC      = B5 33   → 0x33B5 (düşük bayt önce geldi)
```

Kendi CRC fonksiyonu `01 03 02 12 34` baytları için `0x33B5` üretiyorsa doğru yolda demektir. Bu, projenin ilk kontrol noktası.

Programın yapacağı 10 adım

Gönderen taraf:

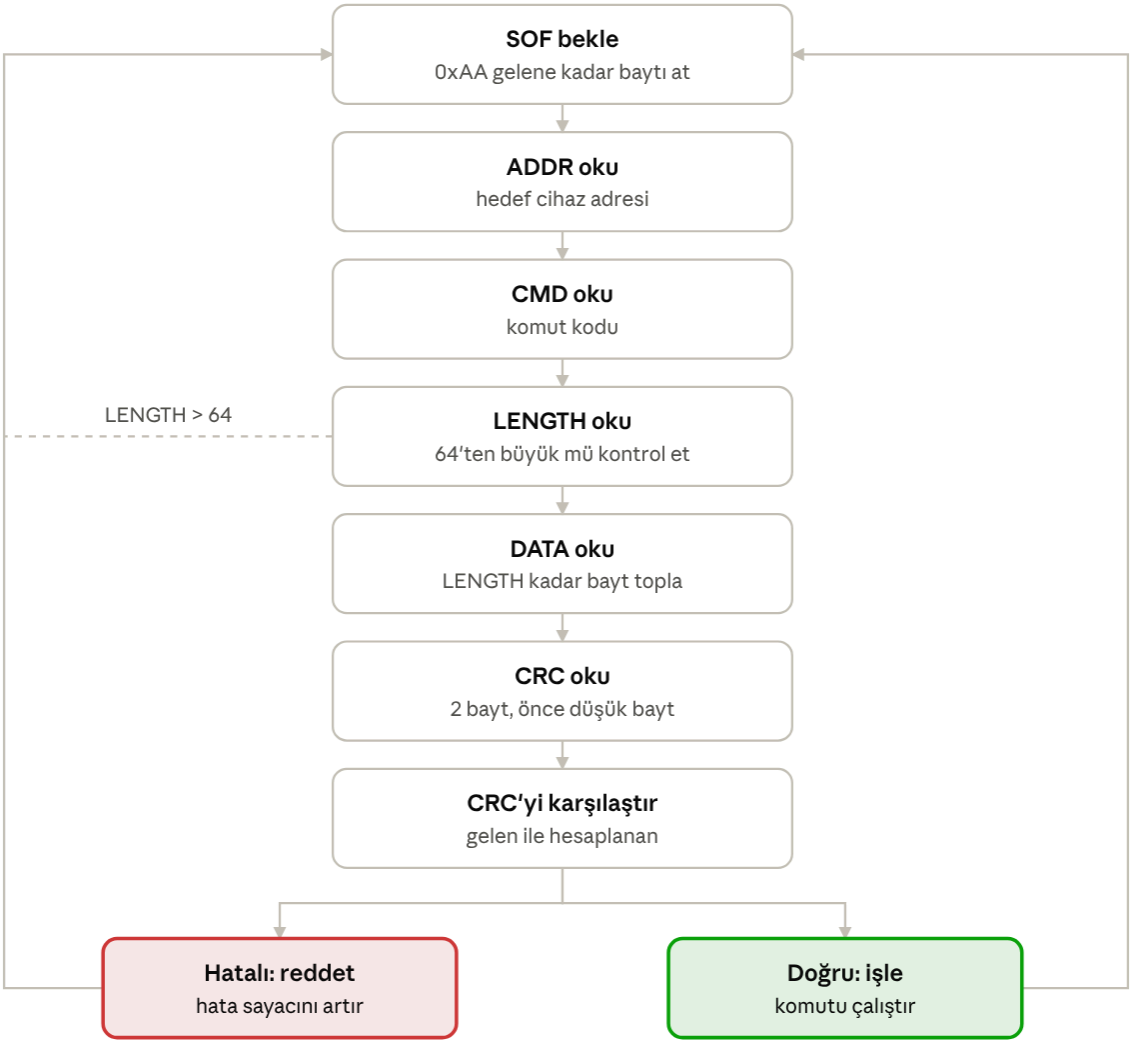
1. **Paketi oluştur.** `packet_build()` alanları sırayla bir bayt dizisine yazar, CRC'yi hesaplayıp sona ekler, yazılan bayt sayısını döndürür. Struct `memcpy` ile kopyalanmaz, alanlar tek tek yazılır.
2. **Buffer'a koy.** Üretilen baytlar "hat" yerine geçen 4. haftadaki ring buffer'a itilir. Gerçek hayatta burası seri port ya da socket olacak.

Alıcı taraf:

3. **Paketi oku.** Ring buffer'dan çekilen baytlar parser'a teker teker verilir: `parser_feed(&p, bayt)`. Hepsini birden vermek yasak, çünkü gerçek hatta veri parça parça gelir.
4. **SOF ara.** `0xAA` görülene kadar gelen her şey atılır. Hat ilk açıldığında gelen baytlar çöp olabilir.
5. **LENGTH oku.** 64'ten büyükse paket geçersizdir ve parser SOF aramaya döner. Bu kontrol yapılmazsa bozuk bir LENGTH baytı buffer taşmasına yol açar; projedeki en tehlikeli hata budur.
6. **DATA'yı ayır.** LENGTH kadar bayt paket struct'ının `data` dizisine toplanır.
7. **CRC hesapla.** ADDR'dan DATA sonuna kadar olan baytların CRC'si alıcıda yeniden hesaplanır.
8. **CRC'yi karşılaştır.** Gelen iki CRC baytı (önce düşük) 16 bitlik sayıya birleştirilip hesaplananla karşılaştırılır.
9. **Hatalı paketi reddet.** Hata sayacı artırılır, parser SOF aramaya döner. Program çökmez, durmaz.
10. **Geçerli paketi işle.** Komut tablosuna göre çalıştırılır ve ekrana yazılır, örneğin `[OK] addr=1 cmd=VALUE_WRITE value=4660`.

Parser'ın durum makinesi

Parser her baytta bir adım ilerler, sonunda hep başa döner



packet parser durum makinesi · 7 durum, 2 sonuç

Parser'ın tek görevi "şu an hangi alanı bekliyorum?" sorusunun cevabını hatırlamaktır. Her yeni baytta mevcut duruma bakılır, bayt işlenir ve sonraki duruma geçilir; bu yüzden veri birer birer de gelse, yüzer yüzer de gelse sonuç aynı olur.

Ödev detayları

Dosya yapısı

```
proje1/
├── Makefile
├── README.md
├── include/   crc16.h  ringbuf.h  packet.h  parser.h
├── src/       crc16.c  ringbuf.c  packet.c  parser.c  main.c
└── tests/     test_parser.c
```

Önerilen arayüz

Fonksiyonların içi tamamen kendisine ait; arayüz sadece yön göstermek için.

```
#define PKT_SOF       0xAA
#define PKT_MAX_DATA  64

typedef struct {
    uint8_t addr;
    uint8_t cmd;
    uint8_t len;
    uint8_t data[PKT_MAX_DATA];
} packet_t;

typedef enum {
    ST_WAIT_SOF, ST_ADDR, ST_CMD, ST_LEN,
    ST_DATA, ST_CRC_LO, ST_CRC_HI
} parser_state_t;

typedef enum {
    PARSE_BUSY,      /* paket henüz tamamlanmadı */
    PARSE_OK,        /* geçerli paket hazır */
    PARSE_ERR_CRC,
    PARSE_ERR_LEN
} parse_result_t;

typedef struct {
    parser_state_t state;
    packet_t       pkt;
    uint8_t        idx;      /* DATA'da kaçıncı bayttayız */
    uint16_t       crc_rx;   /* gelen CRC */
    uint32_t       ok_count, err_crc, err_len;
} parser_t;

uint16_t       crc16_modbus(const uint8_t *buf, size_t len);
size_t         packet_build(const packet_t *p, uint8_t *out, size_t
out_size);
void           parser_init(parser_t *ps);
parse_result_t parser_feed(parser_t *ps, uint8_t byte);
```

Test senaryoları

Tüm CRC değerleri doğrulandı; testlere doğrudan konabilir.

#	Senaryo	Girdi	Beklenen
1	Geçerli paket	`AA 01 03 02 12 34 B5 33`	OK, değer 4660
2	Verisiz paket	`AA 01 01 00 21 90`	OK, PING
3	Bozuk CRC	`AA 01 03 02 12 34 B5 34`	`ERR_CRC`, sayaç 1
4	Baştaki çöp	`00 FF 13 AA 01 01 00 21 90`	Çöp atlanır, OK
5	Arka arkaya iki paket	`AA 01 01 00 21 90 AA 01 04 00 22 C0`	İki OK
6	Parça parça gelen paket	Önce `AA 01 03`, sonra `02 12 34 B5 33`	OK
7	Çok büyük LENGTH	`AA 01 03 C8` + `AA 01 01 00 21 90`	Önce `ERR_LEN`, sonra OK
8	Veri içinde 0xAA	`AA 01 03 02 AA 10 C7 28`	OK; verideki AA yeni paket sanılmaz
9	Bilinmeyen komut	`AA 01 7F 00 00 30`	CRC doğru, "bilinmeyen komut"

8 numaralı test durum makinesinin neden doğru yaklaşım olduğunu gösteriyor: parser DATA durumundayken `0xAA`'ya özel anlam vermez, sadece bayt sayar.

Kabul kriterleri

- [ ] `make` tek komutla derliyor; `-Wall -Wextra -Werror` ile hiç uyarı yok
- [ ] `make test` dokuz senaryonun hepsini geçiyor
- [ ] `-fsanitize=address,undefined` ile çalıştırınca hata yok, valgrind temiz
- [ ] `crc16_modbus("123456789")` sonucu `0x4B37`
- [ ] README'de paket formatı, komut tablosu ve derleme talimatı var
- [ ] Kod incelemesinde her durum geçişini açıklayabiliyor

Sık yapılan hatalar

- **Struct'ı `memcpy` ile buffer'a kopyalamak:** Padding baytları ve bayt sırası yüzünden karşı taraf yanlış okur.
- **LENGTH sınırını kontrol etmemek:** Doğrudan buffer taşması.
- **CRC baytlarının sırasını karıştırmak:** Önce düşük bayt gelir.
- **Hatadan sonra parser'ı sıfırlamamak:** Bir bozuk paket sonraki bütün paketleri bozar.
- **`char` kullanmak:** `0xAA`, signed `char`'da negatif sayıya dönüşür; her yerde `uint8_t`.

Bonus görevler

- **Adres filtresi:** Sadece kendi adresine ve `0xFF` yayın adresine gelenleri işlesin.
- **Yanıt paketi:** PING gelince PONG paketi üretsin.
- **Hex dump:** Buffer'ı `AA 01 03 ...` biçiminde yazdıran hata ayıklama fonksiyonu.
- **Dosyadan besleme:** `./parser kayit.bin` ile bir dosyadaki baytları parser'a veren araç.
- **Fuzz testi:** Bir milyon rastgele bayt beslendiğinde program çökmesin, sanitizer hata vermesin.
- **İleri seviye yeniden senkronizasyon:** CRC hatasından sonra, sahte SOF'tan hemen sonraki bayttan itibaren yeniden `0xAA` arayarak arada kalmış geçerli bir paketi kurtarsın.

Kod incelemesinde sorulacak sorular

1. SOF neden CRC hesabına dahil edilmedi?
2. LENGTH kontrolü olmasaydı hangi satırda ne bozulurdu?
3. Veri içinde `0xAA` olunca parser neden şaşırmıyor?
4. CRC hatalı bir paketten sonra parser hangi durumda?
5. Struct neden alan alan yazıldı da `memcpy` kullanılmadı?
6. Parser'ın bayt bayt beslenmesi gerçek bir seri porta ya da ağ bağlantısına bağlanırken ne kazandıracak?

Kaynaklar

Ana kaynak Beej's Guide; diğerleri konuyu farklı açıdan görmek, denemek ve pekiştirmek için. Her konu için: Beej'den oku, learn-c.org'da dene, Exercism'den bir alıştırma çöz.

Kaynak	Ne için	Hafta
Beej's Guide to C Programming	Ücretsiz ana okuma kaynağı	1-4
learn-c.org	Tarayıcıda hızlı deneme, kısa interaktif dersler	1-3
Exercism C track	Testli alıştırmalar, kendi bilgisayarında komut satırından	1-4
CS50x	Harvard'ın ücretsiz dersi; 1. ve 2. hafta videoları C temelleri	1-2
CS50x Week 4: Memory	Pointer, malloc ve valgrind anlatımı	3

Kaynak	Ne için	Hafta
Beej's Quick Guide to GDB	Debugger'a kısa giriş	1-4
Python Tutor (C görselleştirici)	Stack, heap ve pointer'ları adım adım görmek	2-3
Compiler Explorer	Struct boyutları, padding, bit işlemlerinin derlenmiş hali	3-4
CRC hesaplayıcı	CRC-16/MODBUS sonuçlarını doğrulamak	4

Kağıt ve kalem de kaynak sayılır: sayı dönüşümleri, pointer diyagramları ve durum makinesi önce kağıtta çizilmeli.